In [1]:
```
import pandas as pd

# Load dataset
df = pd.read_csv("C:/Users/Govin/Downloads/loan_approval_dataset.csv")
df.head()  # Show first few rows
```

Out[1]:

| | loan_id | no_of_dependents | education | self_employed | income_annum | loan_amount | loa |
|---|---|---|---|---|---|---|---|
| **0** | 1 | 2 | Graduate | No | 9600000 | 29900000 | |
| **1** | 2 | 0 | Not Graduate | Yes | 4100000 | 12200000 | |
| **2** | 3 | 3 | Graduate | No | 9100000 | 29700000 | |
| **3** | 4 | 3 | Graduate | No | 8200000 | 30700000 | |
| **4** | 5 | 5 | Not Graduate | Yes | 9800000 | 24200000 | |

In [2]:
```
# Check shape and data types
print("Dataset shape:", df.shape)
df.dtypes
```

```
Dataset shape: (4269, 13)
```

Out[2]:
```
loan_id                     int64
 no_of_dependents           int64
 education                 object
 self_employed             object
 income_annum               int64
 loan_amount                int64
 loan_term                  int64
 cibil_score                int64
 residential_assets_value   int64
 commercial_assets_value    int64
 luxury_assets_value        int64
 bank_asset_value           int64
 loan_status               object
dtype: object
```

In [3]:
```
# Strip column names of spaces
df.columns = df.columns.str.strip()

# Check for nulls
df.isnull().sum()
```

Out[3]:
```
loan_id                     0
no_of_dependents            0
education                   0
self_employed               0
income_annum                0
loan_amount                 0
loan_term                   0
cibil_score                 0
residential_assets_value    0
commercial_assets_value     0
luxury_assets_value         0
bank_asset_value            0
loan_status                 0
dtype: int64
```

In [4]:
```
from sklearn.preprocessing import LabelEncoder

df_encoded = df.copy()
le = LabelEncoder()

# Clean and encode categorical columns
df_encoded['education'] = le.fit_transform(df_encoded['education'].str.strip())
df_encoded['self_employed'] = le.fit_transform(df_encoded['self_employed'].str.stri
df_encoded['loan_status'] = df_encoded['loan_status'].str.strip().map({'Approved':

df_encoded.head()
```

Out[4]:

| | loan_id | no_of_dependents | education | self_employed | income_annum | loan_amount | loa |
|---|---|---|---|---|---|---|---|
| **0** | 1 | 2 | 0 | 0 | 9600000 | 29900000 | |
| **1** | 2 | 0 | 1 | 1 | 4100000 | 12200000 | |
| **2** | 3 | 3 | 0 | 0 | 9100000 | 29700000 | |
| **3** | 4 | 3 | 0 | 0 | 8200000 | 30700000 | |
| **4** | 5 | 5 | 1 | 1 | 9800000 | 24200000 | |

In [5]:
```
# New features
df_encoded['debt_income_ratio'] = df_encoded['loan_amount'] / df_encoded['income_an
df_encoded['monthly_emi'] = df_encoded['loan_amount'] / df_encoded['loan_term']

# View updated table
df_encoded.head()
```

Out[5]:

| | loan_id | no_of_dependents | education | self_employed | income_annum | loan_amount | loa |
|---|---|---|---|---|---|---|---|
| **0** | 1 | 2 | 0 | 0 | 9600000 | 29900000 | |
| **1** | 2 | 0 | 1 | 1 | 4100000 | 12200000 | |
| **2** | 3 | 3 | 0 | 0 | 9100000 | 29700000 | |
| **3** | 4 | 3 | 0 | 0 | 8200000 | 30700000 | |
| **4** | 5 | 5 | 1 | 1 | 9800000 | 24200000 | |

In [6]:

```
from sklearn.model_selection import train_test_split

X = df_encoded.drop(['loan_id', 'loan_status'], axis=1)
y = df_encoded['loan_status']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

In [7]:

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

models = {
    "Random Forest": RandomForestClassifier(random_state=42),
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Gradient Boosting": GradientBoostingClassifier()
}

results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)
    results.append((name, acc))
    print(f"📊 {name} Report")
    print("Accuracy:", acc)
    print("Confusion Matrix:\n", confusion_matrix(y_test, preds))
    print("Classification Report:\n", classification_report(y_test, preds))
    print("-" * 40)
```

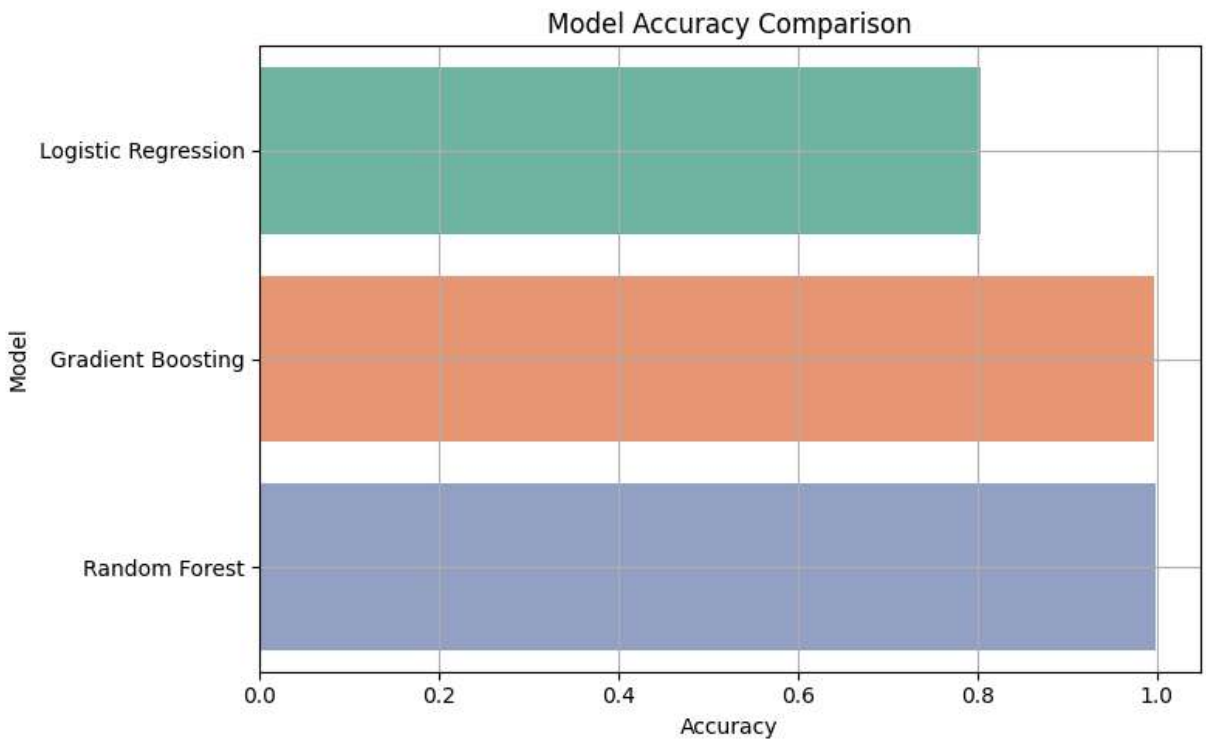
```
📊 Random Forest Report
Accuracy: 0.9988290398126464
Confusion Matrix:
 [[317   1]
 [  0 536]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       318
           1       1.00      1.00      1.00       536

    accuracy                           1.00       854
   macro avg       1.00      1.00      1.00       854
weighted avg       1.00      1.00      1.00       854

----------------------------------------
📊 Logistic Regression Report
Accuracy: 0.8032786885245902
Confusion Matrix:
 [[199 119]
 [ 49 487]]
Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.63      0.70       318
           1       0.80      0.91      0.85       536

    accuracy                           0.80       854
   macro avg       0.80      0.77      0.78       854
weighted avg       0.80      0.80      0.80       854

----------------------------------------
📊 Gradient Boosting Report
Accuracy: 0.9976580796252927
Confusion Matrix:
 [[317   1]
 [  1 535]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       318
           1       1.00      1.00      1.00       536

    accuracy                           1.00       854
   macro avg       1.00      1.00      1.00       854
weighted avg       1.00      1.00      1.00       854

----------------------------------------
```

In [8]:

```
import matplotlib.pyplot as plt
import seaborn as sns

results_df = pd.DataFrame(results, columns=["Model", "Accuracy"]).sort_values(by="A

plt.figure(figsize=(8, 5))
sns.barplot(data=results_df, x="Accuracy", y="Model", palette="Set2")
```

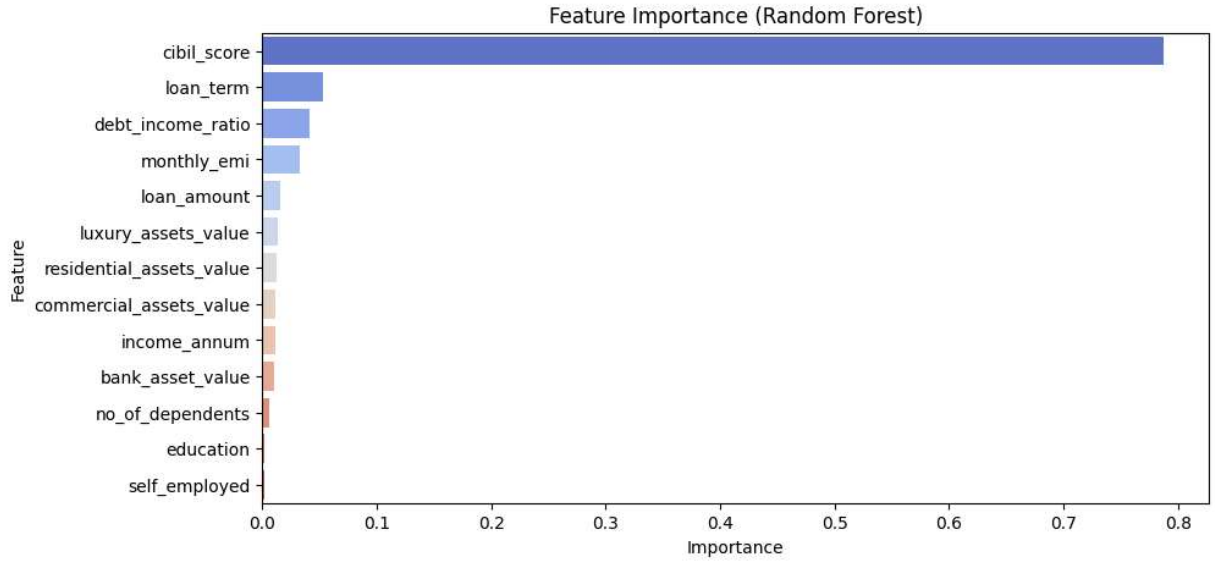
```
plt.title("Model Accuracy Comparison")
plt.xlabel("Accuracy")
plt.ylabel("Model")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
C:\Users\Govin\AppData\Local\Temp\ipykernel_30428\3747916264.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.1
4.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(data=results_df, x="Accuracy", y="Model", palette="Set2")
```

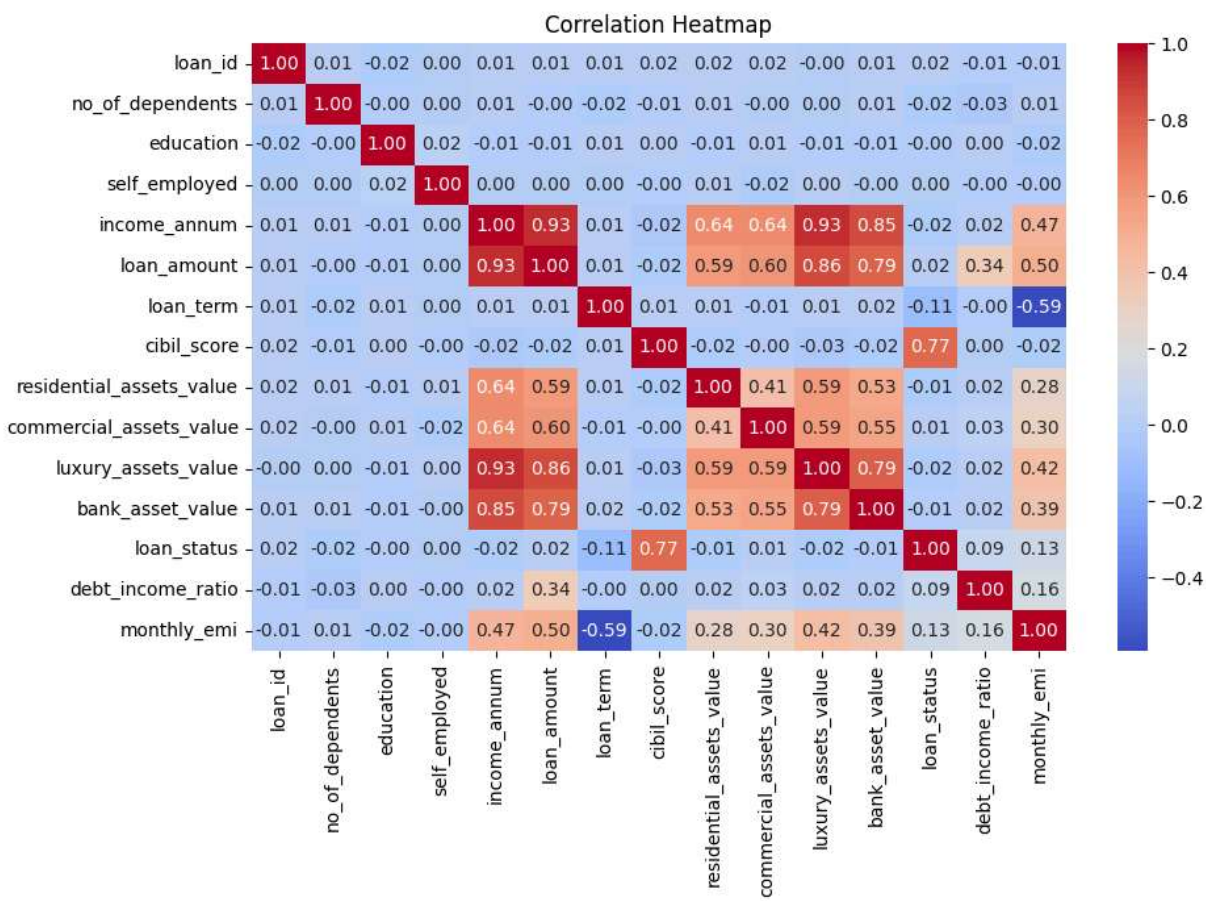



In [9]:
```
importances = models["Random Forest"].feature_importances_
features = X.columns

feat_df = pd.DataFrame({"Feature": features, "Importance": importances})
feat_df = feat_df.sort_values(by="Importance", ascending=False)

plt.figure(figsize=(10, 5))
sns.barplot(data=feat_df, x="Importance", y="Feature", palette="coolwarm")
plt.title("Feature Importance (Random Forest)")
plt.show()
```

```
C:\Users\Govin\AppData\Local\Temp\ipykernel_30428\2993136634.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.1
4.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(data=feat_df, x="Importance", y="Feature", palette="coolwarm")
```

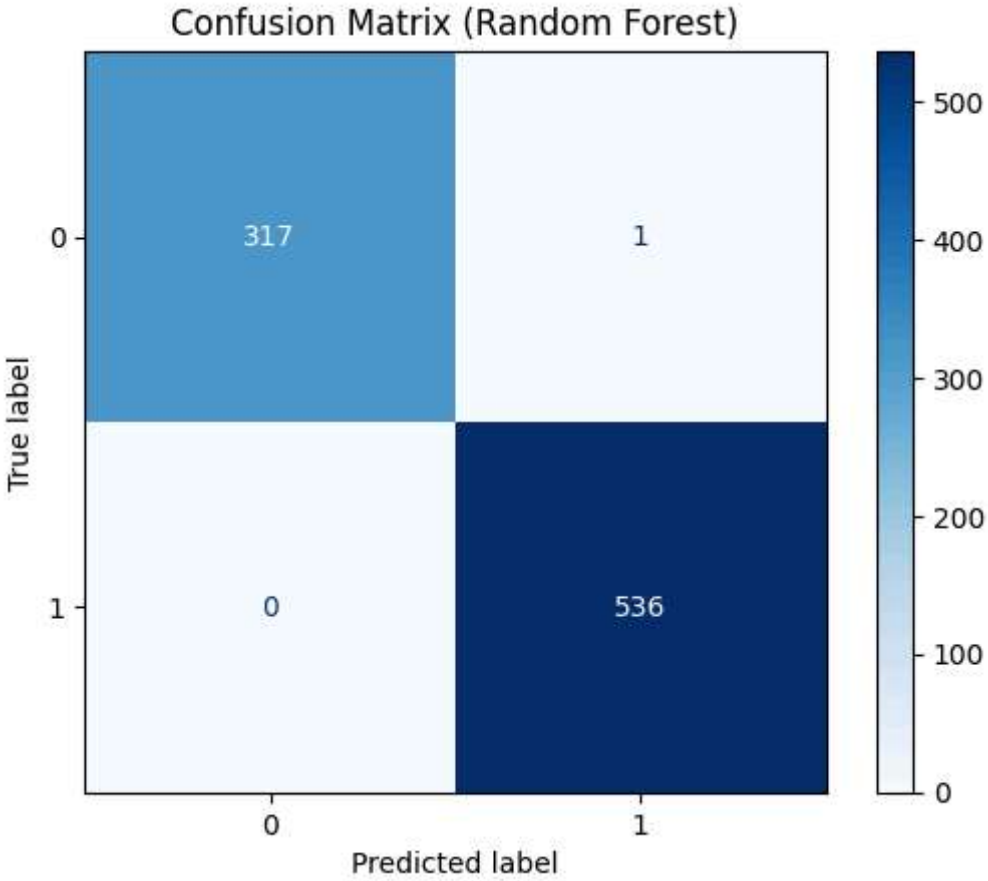



In [10]:
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
sns.heatmap(df_encoded.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```



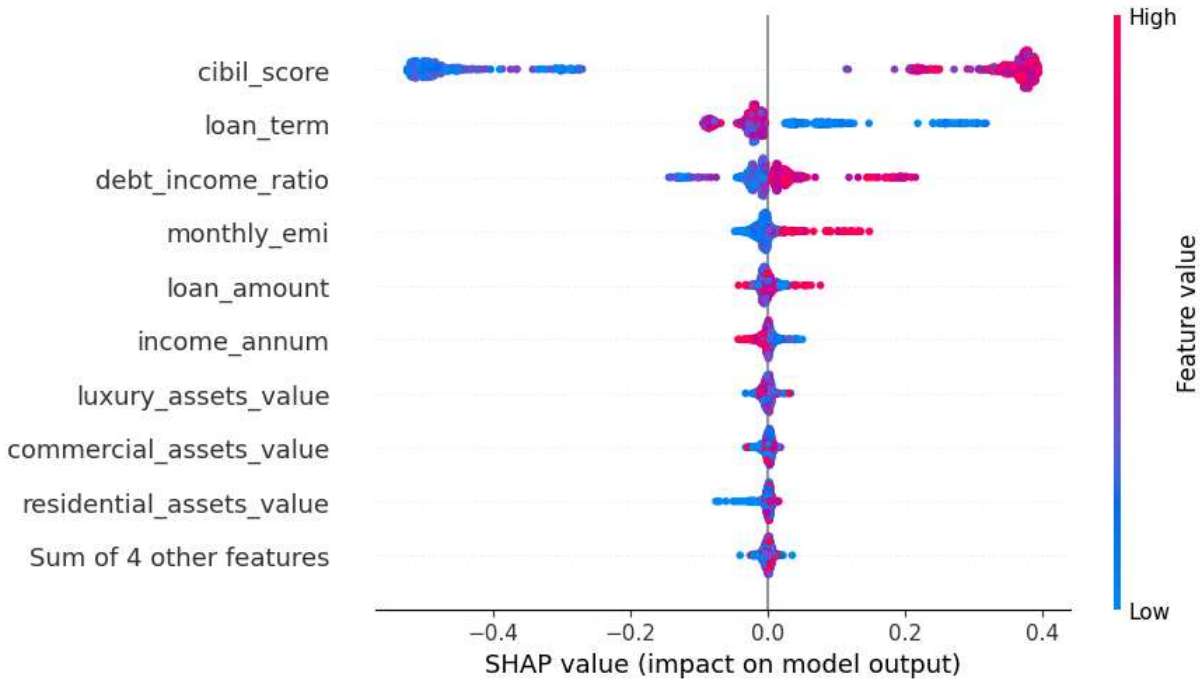

In [11]:
```
from sklearn.metrics import ConfusionMatrixDisplay
```

```
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

ConfusionMatrixDisplay.from_estimator(model, X_test, y_test, cmap='Blues')
plt.title("Confusion Matrix (Random Forest)")
plt.show()
```




In [13]:
```
import shap

# Use the same model you trained
explainer = shap.Explainer(model, X_test)

# Get SHAP values
shap_values = explainer(X_test)

# Plot beeswarm for class 1 (Approved)
shap.plots.beeswarm(shap_values[:, :, 1])  # or shap_values[..., 1]
```

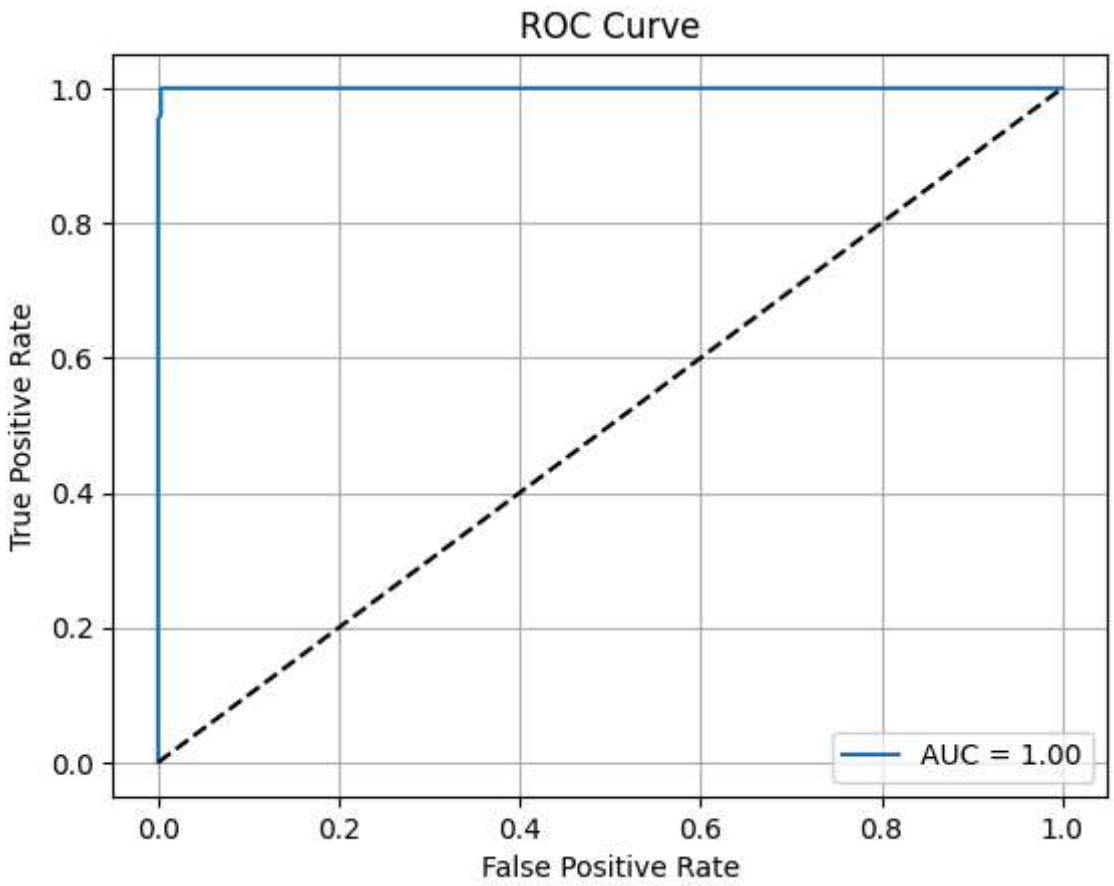
```
96%|=================== | 1636/1708 [00:16<00:00]
```




In [14]:
```
from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(model, X, y, cv=5)
print("Cross-validation scores:", cv_scores)
print("Mean CV Accuracy:", cv_scores.mean())
```

```
Cross-validation scores: [0.9941452  0.99531616 0.99765808 0.99648712 0.99765533]
Mean CV Accuracy: 0.9962523782983876
```

In [15]:
```
from sklearn.metrics import roc_curve, auc

y_proba = model.predict_proba(X_test)[:, 1]
fpr, tpr, _ = roc_curve(y_test, y_proba)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}")
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.grid()
plt.show()
```


ROC Curve
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
AUC = 1.00